



DOCUMENTAÇÃO OFICIAL DO PROJETO

ArkGO

Guia completo do produto, operação, arquitetura e próximos passos

Aplicativo de intermediação logística urbana para conectar clientes a motoboys e outros veículos de entrega.

Versão do documento: 1.0

Data de referência: 17 de julho de 2026

Local do projeto: UrbGO/urbgo

Nome técnico do pacote: arkgo

1. Resumo executivo

O ArkGO é um aplicativo móvel de logística urbana. O cliente informa origem, destino e categoria do veículo; o sistema calcula o preço, encontra um motoboy compatível e permite acompanhar a entrega. O motoboy fica online, recebe corridas próximas, aceita uma delas, navega até a coleta e confirma a conclusão. A operação é apoiada por notificações, localização, chat, avaliações, carteira e painel de indicadores.

Leitura rápida do estado: o núcleo do MVP está implementado em Flutter, com autenticação, fluxos de cliente e motoboy, acompanhamento em tempo real, chat, avaliações, carteira PIX, roteamento e funções administrativas. Ainda existem itens importantes de produto, qualidade e produção listados neste documento.

Objetivo do negócio

Reduzir a dificuldade de contratar entregas urbanas, dando previsibilidade ao cliente e mais oportunidades de trabalho ao motoboy. O produto também cria uma base operacional mensurável: corridas, distância, tempo, tráfego, avaliações, saldo e eficiência de rotas.

Decisões importantes

- Um único aplicativo atende os perfis cliente e motoboy, escolhendo o fluxo correto após o login.
- O aplicativo conversa com um backend hospedado, que guarda usuários, entregas e eventos.
- Recursos sensíveis, como localização, documentos, notificações e pagamentos, dependem de consentimento e configuração correta.
- O nome comercial usado no código é ArkGO, embora o diretório de trabalho seja UrbGO/urbgo.

2. O que é o ArkGO

Explicação para quem não é da área de tecnologia

Pense no ArkGO como uma central digital de entregas. O cliente pede uma corrida pelo celular; o aplicativo calcula uma estimativa; motoboys disponíveis recebem a oportunidade; um deles aceita; ambos acompanham o andamento. O “painel” e os serviços de bastidores fazem o trabalho que não aparece na tela: salvar informações, enviar alertas, calcular rotas e confirmar pagamentos.

Quem usa

Perfil	Necessidade	Resultado esperado
Cliente	Enviar um item com praticidade e visibilidade.	Pedido criado, preço compreendido, entrega acompanhada e avaliada.
Motoboy	Encontrar corridas compatíveis e controlar ganhos.	Corrida aceita, rota executada, saldo e histórico atualizados.
Administrador / negócio	Entender a operação e tomar decisões.	Indicadores, eficiência de roteamento e visão para investidores.

O que o produto não é

O repositório não indica um marketplace web independente nem uma operação de frota própria. Ele representa principalmente um aplicativo móvel e um backend Supabase. A disponibilidade comercial, cobertura geográfica, contratos, regras fiscais, suporte humano e políticas operacionais precisam ser definidos fora do código.

3. Como a operação funciona

CLIENTE → informa coleta/destino → preço e rota → cria entrega ↓ backend filtra categoria, distância e disponibilidade ↓ MOTOBOY ← recebe alerta ← visualiza corrida → aceita (sujeito a saldo/regras) ↓ desloca-se → confirma coleta → entrega → confirma conclusão ↓ CLIENTE acompanha localização/ETA → conversa → avalia ↓ histórico, carteira, métricas e relatório

Estados de uma entrega

Estado	O que significa na prática
pending	Pedido criado, aguardando motoboy.
accepted	Motoboy assumiu a corrida.
pickup / chegada	Etapa de deslocamento e coleta; os nomes exatos devem ser conferidos no banco em produção.
in_transit	Item está a caminho do destino.
completed	Entrega concluída.
cancelled / expired	Cancelada por regra ou expirada após ficar pendente.

A implementação possui fluxo de confirmação de coleta, conclusão, abandono e timeout; a nomenclatura completa é distribuída entre o app e as migrações SQL.

4. Funcionalidades por perfil

Cliente

- Cadastro, login, confirmação de e-mail e recuperação de senha.
- Onboarding inicial e aceite de privacidade/termos.
- Criação de entrega com endereços, geocodificação, CEP e categoria do veículo.
- Prévia e composição do preço, incluindo distância, tráfego, pedágio e ida/volta quando aplicável.
- Acompanhamento em mapa, posição do motoboy e previsão de chegada.
- Histórico, perfil, cancelamento dentro das regras e upload de foto de entrega.
- Chat em tempo real com o motoboy e avaliação de 1 a 5 estrelas.
- Notificações de mudanças relevantes.

Motoboy

- Cadastro com dados pessoais, endereço, veículo e documentos para análise.
- Estado "aguardando liberação" quando o cadastro ainda não foi aprovado.
- Alternância online/offline e busca de corridas compatíveis por categoria, distância e proximidade.
- Aceite atômico da corrida, com proteção contra dois motoboys aceitarem o mesmo pedido.
- Execução da corrida, confirmação de coleta, navegação externa e abandono quando permitido.
- Localização em segundo plano durante entrega ativa e notificação persistente da corrida.
- Carteira, recarga PIX, confirmação de pagamento, transações e painel de ganhos.
- Histórico, perfil, reputação, avaliações e chat.

Administração e inteligência

- Dashboard de métricas operacionais e feed de eventos.
- Tela de apresentação para investidores.
- Telemetria de rotas: ETA base, ETA com tráfego/IA, tempo real, fonte da rota e eficiência.
- Relatório semanal gerado em PDF e armazenado no Storage do Supabase.

5. Arquitetura explicada para leigos

A arquitetura é formada por três camadas. A primeira é o aplicativo que a pessoa instala. A segunda é o backend, equivalente à central de atendimento e ao arquivo de dados. A terceira são serviços externos especializados, como mapas, Firebase e gateway de pagamento.

Camada	Responsabilidade	Exemplo no projeto
Aplicativo	Telas, botões, validações, mapas e interação com o usuário.	lib/features, Flutter/Dart.
Dados e autenticação	Usuários, entregas, regras de acesso, atualizações em tempo real e arquivos.	Supabase Auth, PostgreSQL, Realtime, Storage.
Funções de backend	Executar tarefas protegidas e integrar APIs sem expor segredos no celular.	supabase/functions, Deno/TypeScript.
Serviços externos	Rotas, tráfego, CEP, push e PIX.	OSRM, Google Routes, ViaCEP, Firebase FCM e Pagar.me.

Organização interna do aplicativo

O código segue uma separação por domínio: autenticação, cliente, motoboy, entrega e componentes compartilhados. “Repository” é a parte que busca ou salva dados; “service” integra uma capacidade externa; “screen” desenha uma tela; “provider” distribui estado para as telas.

O Riverpod mantém o estado organizado, o GoRouter controla as rotas e os redirecionamentos de acesso, e o Realtime atualiza telas sem exigir que o usuário recarregue tudo manualmente.

6. Stack tecnológica

Área	Tecnologia	Uso
Aplicativo	Flutter + Dart	Base multiplataforma para Android e iOS.
Estado	Riverpod / riverpod_generator	Estado reativo e injeção de dependências.
Navegação	GoRouter	Rotas, deep links e proteção por sessão/perfil.
Backend	Supabase	Auth, PostgreSQL, Realtime, Storage e Edge Functions.
Edge backend	Deno + TypeScript	Funções serverless de roteamento, PIX, push e relatórios.
Mapas/localização	flutter_map, latlong2, geolocator, OSRM e Google Routes	Mapa, GPS, cálculo e decisão de rota.
Push	Firebase Cloud Messaging + notificações locais	Alertas de corrida, andamento e recarga.
Pagamentos	PIX via funções e Pagar.me	Recarga da carteira do motoboy.
UI e mídia	Google Fonts, image_picker, image_cropper, cached_network_image	Tipografia, documentos, fotos e imagens.
Plataformas	Android Gradle 8.14 / Firebase; iOS Xcode project	Empacotamento e integração nativa.

Dependências declaradas e versões-base estão em pubspec.yaml. O SDK declarado é Dart ^3.11.1; o ambiente observado nesta documentação reportou Dart 3.12.0.

7. Mapa do projeto

Caminho	Conteúdo
lib/app	Inicialização visual, tema, transições e roteamento.
lib/core	Constantes, serviços gerais, validações, notificações, GPS, pagamentos e integrações.
lib/data	Modelos de rota, cálculo de tarifa e repositório de rotas.
lib/features/auth	Cadastro, login, onboarding e aprovação do motoboy.
lib/features/client	Criação, acompanhamento, histórico, chat e avaliações do cliente.
lib/features/motoboy	Disponibilidade, corrida ativa, carteira, ganhos, histórico e perfil.
lib/features/admin	Dashboard e pitch de investidores.
lib/features/shared	Modelos e widgets reutilizados.
supabase/migrations	Evolução versionada do banco, RLS, triggers, funções e regras.
supabase/functions	10 funções Edge, incluindo rota, PIX, push, analytics e relatórios.
assets	Logos, imagens de veículos, áudio de alerta e termos PDF.
android / ios	Configurações nativas, permissões, deep links, ícones e Firebase.
docs	Análises, SQL auxiliar, relatório anterior e esta documentação.
build_android.sh / run.sh	Automação de execução local e build Android.

Local do projeto: `/home/appmoove/Imagens/projetos/UrbGO/urbgo`. O repositório tinha alterações locais não commitadas na data de referência: quatro arquivos Dart modificados, uma migração nova e `test/db_test.dart` novo. Elas foram preservadas.

8. Dados, integrações e configurações

Dados principais

As migrações indicam entidades para usuários, motoboys, entregas, notificações por proximidade, mensagens, avaliações, transações, regras de preço, telemetria de rotas, cache de rota e armazenamento de documentos/fotos. O banco usa políticas de segurança por linha (RLS) e funções SQL para operações atômicas.

Funções Edge

Função	Finalidade
routing-decision	Consulta OSRM e, quando necessário, Google Routes com tráfego.
gemini-routing	Integração de decisão/roteamento inteligente.
create-pix-charge	Cria cobrança PIX e registra recarga pendente.
check-pix-status	Consulta o status de uma cobrança.
pix-webhook	Recebe confirmação do gateway e credita a carteira com proteção contra duplicidade.
send-push	Envia notificações por Firebase FCM HTTP v1.
simulate-recharge	Recarga simulada; somente desenvolvimento.
analytics-metrics	Expõe métricas para o dashboard.
generate-report	Gera e armazena relatório semanal em PDF.
weekly-report-cron	Dispara o relatório programado.

Variáveis de ambiente

Variável	Onde é usada	Cuidados
SUPABASE_URL	App e Edge Functions.	URL pública do projeto; não substituir por URL de outro ambiente.
SUPABASE_ANON_KEY	App.	Chave pública, protegida pelas políticas RLS.
ORS_API_KEY	Scripts/configuração de rota.	Chave de serviço; revisar se o fluxo atual ainda depende dela.
GOOGLE_MAPS_API_KEY / GOOGLE_MAPS_KEY	Mapa/rotas.	Restringir por API, app e ambiente; nomes devem ser padronizados.
FIREBASE_SERVICE_ACCOUNT	Push no backend.	Segredo crítico; nunca ir para o aplicativo.
PAGARME_WEBHOOK_SECRET	Assinatura do webhook PIX.	Obrigatório em produção.
DASHBOARD_TOKEN e service role	Relatórios/painel.	Somente servidor; não expor ao cliente.

Atenção: `.env` é ignorado pelo Git, mas precisa existir localmente. Não copie chaves reais para documentação, chamados, prints ou commits. O código de webhook indica comportamento permissivo quando o segredo não está configurado; em produção isso deve ser tratado como bloqueador.

9. Como executar e desenvolver

Pré-requisitos

- Flutter SDK compatível com Dart 3.11 ou superior.
- Android Studio/SDK para Android; Xcode e macOS para iOS.
- Projeto Supabase configurado e migrações aplicadas na ordem.
- Projeto Firebase com `google-services.json` para Android e configuração equivalente para iOS.
- Arquivo `.env` preenchido com ambiente de desenvolvimento.

Executar localmente

```
flutter pub get ./run.sh
```

O `run.sh` carrega variáveis do `.env` e inicia flutter run. Também é possível executar o comando diretamente com `--dart-define`.

Verificar o código

```
flutter analyze lib/ flutter test
```

Há testes de cálculo de tarifa e testes placeholder. O ambiente usado para este documento não possuía o executável Flutter disponível no PATH, portanto uma execução completa de análise/build deve ser feita em uma máquina com Flutter instalado.

Gerar Android

```
./build_android.sh apk ./build_android.sh bundle ./build_android.sh both
```

O script valida a análise estática e gera APK e/ou AAB release. O resultado esperado fica em `build/app/outputs/...`. Existe também um `arkgo.apk` na raiz, mas ele deve ser tratado como artefato existente, não necessariamente como build atual.

10. Publicação e operação

Checklist de publicação

1. Separar ambientes dev, homologação e produção.
2. Aplicar migrações Supabase em ordem e confirmar RLS, triggers, cron e Storage.
3. Configurar chaves de mapas, Firebase, gateway PIX e secrets das Edge Functions.

4. Testar cadastro dos dois perfis, aprovação do motoboy, criação/aceite/conclusão, notificações, chat, avaliação e recarga.
5. Testar GPS com tela bloqueada, app em segundo plano, bateria restrita e ausência de rede.
6. Gerar AAB assinado, validar versão/build number e instalar em aparelho real.
7. Publicar política de privacidade, termos e contato de suporte.
8. Monitorar logs, falhas, cobranças duplicadas, entregas presas e consumo das APIs.

Rotina operacional recomendada

Frequência	Verificação
Diária	Entregas pendentes antigas, falhas de push, webhooks PIX, erros de rota e saldo.
Semanal	Relatório de eficiência, avaliações, corridas canceladas, tempo médio e custos de APIs.
Mensal	Atualizações de dependências, revisão de permissões, chaves, backups, RLS e custos.
A cada release	Testes em aparelhos reais Android/iOS, permissões, deep links, notificações e regressão dos fluxos.

Diagnóstico rápido

Sintoma	Primeira investigação
Não aparecem corridas	Online do motoboy, localização, categoria, aprovação, raio/proximidade, fila de notificações e RLS.
Cliente não vê posição	Permissão de GPS, foreground service/background mode, Realtime, conexão e consumo de bateria.
Saldo não atualiza	Status da cobrança, webhook, segredo Pagar.me, função atômica e transação.
Push não chega	Token FCM salvo, permissão, channel Android, service account e logs da função.
Rota falha	Coordenadas, OSRM, chave Google, limite/custo e fallback.

11. Segurança, privacidade e conformidade

- Localização é dado sensível do ponto de vista operacional: coletar somente durante a necessidade declarada, explicar o uso e definir retenção.
- Documentos do motoboy, fotos de entrega, telefone e endereço exigem controle de acesso e política de retenção.
- Manter RLS ativa e testada; chaves de service role e conta Firebase ficam somente em secrets do backend.
- O webhook PIX deve rejeitar assinatura ausente ou inválida em produção. O modo permissivo é aceitável apenas em ambiente local controlado.
- Aplicar rate limiting, logs sem dados pessoais desnecessários, rotação de chaves e revisão de permissões de Storage.
- Validar LGPD com responsável jurídico: base legal, consentimento quando aplicável, direitos do titular, encarregado, retenção e atendimento.
- Definir regras comerciais para cancelamento, reembolso, responsabilidade por conteúdo transportado e suporte a incidentes.

Nota de responsabilidade: este documento descreve o que foi encontrado no código. Ele não substitui auditoria de segurança, revisão jurídica, homologação do gateway ou certificação de publicação nas lojas.

12. Estado atual, testes e pontos de atenção

Itens registrados como corrigidos no roadmap

- Proteção de concorrência no aceite de corrida.
- Timeout de corridas pendentes.
- GPS em segundo plano com serviço foreground Android.
- Opção do motoboy desistir de corrida aceita.

Lacunas de produto priorizadas

Prioridade	Item	Impacto
Alta	ETA mais claro para o cliente, chat completo, múltiplos destinos, foto obrigatória e regras de cancelamento.	Confiança, operação e paridade competitiva.
Alta	Paginação, validação de endereço e tratamento de falhas de rede.	Escala e redução de erros.
Média	Ganhos com gráficos, reputação visível, alerta de chegada, deep links e onboarding.	Retenção e eficiência.
Baixa	CI/CD, analytics de produto, mais testes, tema claro, tablets e internacionalização.	Qualidade e expansão.

Qualidade observada

Existem testes unitários para cálculo de tarifa, mas ainda há placeholders de widget e um teste de banco vazio. O roadmap registra ausência de uma suíte automatizada ampla. Antes de escala comercial, recomenda-se cobrir regras de entrega, permissões de acesso, pagamentos idempotentes, notificações, GPS e fluxos de erro.

Riscos técnicos a acompanhar

- Diferença de nomes entre variáveis de chave de mapas (`GOOGLE_MAPS_API_KEY` e `GOOGLE_MAPS_KEY`).
- Dependência de APIs externas com custo, limite e disponibilidade próprios.
- Localização em segundo plano sujeita a políticas do sistema operacional e economia de bateria.
- Arquivos SQL auxiliares e migrações coexistem; precisa haver um procedimento único e documentado de aplicação.
- A função de recarga simulada nunca deve estar acessível em produção.

13. Próximos passos priorizados

1. Preparar produção com segurança: fechar secrets, webhook assinado, RLS/Storage, ambientes, backups e observabilidade.
2. Homologar o fluxo ponta a ponta: dois aparelhos reais, um cliente e um motoboy, incluindo segundo plano, push, chat, cancelamento e recarga.
3. Fortalecer qualidade: remover placeholders, adicionar testes de integração e criar dados de teste reproduzíveis.
4. Concluir experiência essencial: ETA do cliente, validação de endereço, foto/confirmador de entrega, paginação e mensagens de erro offline.
5. Padronizar operação: definir estados oficiais, regras comerciais, SLA de suporte, cobertura de atendimento e indicadores.
6. Publicar com rastreabilidade: versionamento, changelog, CI/CD, assinatura, beta fechado e plano de rollback.
7. Escalar o produto: reputação, ganhos analíticos, múltiplos destinos, meios de pagamento e analytics de produto.

Horizonte	Entrega sugerida	Critério de conclusão
0-2 semanas	Segurança, ambientes, homologação e correção dos bloqueadores.	Fluxo ponta a ponta aprovado em aparelhos reais.

Horizonte	Entrega sugerida	Critério de conclusão
2–6 semanas	ETA, endereços, foto, cancelamento e suite de testes.	Redução de falhas e aceite operacional.
6–12 semanas	Dashboard de ganhos, reputação, observabilidade e CI/CD.	Operação mensurável e releases repetíveis.
Após validação	Multi-stop, pagamentos adicionais, analytics e expansão.	Indicadores de retenção, margem e qualidade sustentáveis.

14. Glossário e referência rápida

Termo	Explicação simples
Flutter	Ferramenta que permite criar Android e iOS a partir de uma base de código.
Backend	Parte que roda fora do celular e guarda regras e dados.
Supabase	Plataforma usada como banco, login, arquivos, tempo real e funções.
RLS	Regra que limita quais linhas cada pessoa pode ler ou alterar.
Realtime	Atualização automática de dados enquanto algo muda.
Edge Function	Pequeno programa executado no servidor sob demanda.
Deep link	Link que abre o app já em uma tela ou entrega específica.
FCM / push	Mensagem enviada ao celular mesmo quando o app não está aberto.
Webhook	Aviso automático que um serviço envia para outro após um evento.
RLS / service role	A primeira protege o acesso normal; a segunda é uma credencial administrativa e deve ficar no servidor.
APK / AAB	APK instala diretamente no Android; AAB é o formato recomendado para a Play Store.
MVP	Primeira versão funcional, suficiente para validar o produto.

Referências do próprio repositório

- [ROADMAP.md](#) — comparação competitiva e prioridades existentes.
- [docs/RELATORIO_COMPLETO_APP_ARKGO.md](#) — relatório anterior de escopo e implementação.
- [docs/COMPETITIVE_ANALYSIS.md](#) — análise competitiva complementar.
- [pubspec.yaml](#) — dependências e versão do aplicativo.
- [build_android.sh](#) e [run.sh](#) — execução e build.
- [supabase/migrations](#) — histórico do modelo e das regras do banco.

Documento gerado a partir da inspeção do repositório ArkGO/UrbGO em 17/07/2026. Atualizar este guia quando houver mudança de fluxo, stack, ambiente, regras de negócio ou status do roadmap.